

Desenvolvimento de Aplicações Web

Lab 05 - Manipulação do DOM

Ano Letivo: 2025/2026

1 Introdução

Neste laboratório vamos aprofundar a **manipulação do DOM (Document Object Model)** numa página criada de raiz, sem depender de ficheiros de laboratórios anteriores.

O **DOM** é a representação estruturada do documento HTML que JavaScript pode manipular. Cada elemento HTML é um **nó** que podemos:

- Selecionar de formas variadas
- Criar e adicionar dinamicamente
- Modificar (conteúdo, atributos, estilos)
- Remover da página
- Responder a eventos do utilizador

Neste laboratório, vamos construir uma **Lista de Compras** simples e interativa, focando inteiramente em manipulação do DOM. Os dados serão guardados num array de strings (não em objetos).

O aspeto final esperado do trabalho a desenvolver está apresentado na Figura 1.



Figura 1: *Aspetto final esperado da Lista de Compras (tema claro)*

2 Objetivos

Neste laboratório iremos:

- Compreender o DOM em profundidade
- Selecionar elementos de múltiplas formas (`querySelector`, `querySelectorAll`)
- Criar e adicionar elementos dinamicamente (`createElement`, `appendChild`)
- Remover elementos do DOM
- Manipular atributos e classes CSS
- Implementar uma **Lista de Compras** funcional com funcionalidades de DOM
- Ordenar e filtrar elementos da página
- Manipular temas e estilos dinamicamente
- Aplicar boas práticas de organização de código JavaScript

3 Pré-requisitos

- Visual Studio Code instalado
- Browser moderno com ferramentas de desenvolvimento
- Conhecimentos básicos de navegação em pastas e ficheiros
- Conhecimentos básicos de HTML, CSS e JavaScript

4 Conceitos-Chave

4.1 O DOM (Document Object Model)

O **DOM** é uma interface de programação que representa o documento HTML como uma **árvore de objetos**. Cada elemento HTML torna-se um **nó** (node) que JavaScript pode manipular.

Estrutura do DOM:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Minha Pagina</title>
  </head>
  <body>
    <h1>Titulo</h1>
    <p>Parágrafo</p>
  </body>
</html>
```

Esta estrutura HTML cria a seguinte árvore DOM:

```
Document
- html
  - head
    - title: "Minha Pagina"
  - body
    - h1: "Titulo"
    - p: "Parágrafo"
```

4.2 Seleção de Elementos

JavaScript oferece várias formas de selecionar elementos HTML:

Método	Descrição
<code>getElementById("id")</code>	Seleciona elemento por ID (único)
<code>querySelector("seletor")</code>	Seleciona primeiro elemento que corresponde
<code>querySelectorAll("seletor")</code>	Seleciona todos os elementos correspondentes
<code>getElementsByClassName("classe")</code>	Seleciona por classe (coleção dinâmica)
<code>getElementsByTagName("tag")</code>	Seleciona por tag (coleção dinâmica)

Tabela 1: Métodos de seleção de elementos

querySelector vs getElementById

querySelector é mais moderno e flexível (aceita qualquer seletor CSS), mas getElementById é ligeiramente mais rápido. Para selecionar por ID, ambos funcionam bem:

```
let elemento1 = document.getElementById('meuId');
let elemento2 = document.querySelector('#meuId'); // Mesmo resultado
```

Exemplos de seletores CSS:

```
// Por ID
let titulo = document.querySelector('#titulo');

// Por classe
let botoes = document.querySelectorAll('.botao');

// Por tag
let paragrafos = document.querySelectorAll('p');

// Seletor complexo
let linkNav = document.querySelector('nav a.ativo');

// Dentro de um elemento específico
let lista = document.getElementById('lista');
let itens = lista.querySelectorAll('li');
```

4.3 Criação e Remoção de Elementos

Para adicionar conteúdo dinamicamente ao DOM:

```
// Criar novo elemento
let novoParagrafo = document.createElement('p');

// Adicionar conteúdo
novoParagrafo.textContent = 'Este é um novo parágrafo';

// Adicionar classes
novoParagrafo.classList.add('destaque');

// Adicionar ao DOM
document.body.appendChild(novoParagrafo);
```

Código 1: Criar e adicionar elementos

Para remover elementos:

```
// Selecionar elemento
let elemento = document.getElementById('remover');

// Remover do DOM
elemento.remove();

// Alternativa (remover filho de um pai)
let pai = document.getElementById('container');
```

```
let filho = document.getElementById('item');
pai.removeChild(filho);
```

Código 2: Remover elementos

4.4 Manipulação de Atributos

Podemos ler e modificar atributos HTML de elementos:

```
let imagem = document.querySelector('img');

// Ler atributo
let fonte = imagem.getAttribute('src');

// Modificar atributo
imagem.setAttribute('src', 'nova-imagem.jpg');
imagem.setAttribute('alt', 'Nova descrição');

// Remover atributo
imagem.removeAttribute('title');

// Para atributos comuns, podemos usar propriedades diretas
imagem.src = 'outra-imagem.jpg';
imagem.alt = 'Outra descrição';
```

4.5 Manipulação de Classes CSS

A propriedade `classList` oferece métodos para manipular classes:

```
let elemento = document.getElementById('caixa');

// Adicionar classe
elemento.classList.add('ativo');

// Remover classe
elemento.classList.remove('inativo');

// Alternar classe (toggle)
elemento.classList.toggle('visivel'); // Adiciona se não existe, remove se existe

// Verificar se tem classe
if (elemento.classList.contains('ativo')) {
    console.log('Elemento esta ativo');
}

// Substituir classe
elemento.classList.replace('antiga', 'nova');
```

Boas Práticas: Classes vs Estilos Inline

Preferir manipular **classes CSS** em vez de estilos inline:

```
// Evitar (estilos inline)
elemento.style.color = 'red';
elemento.style.fontSize = '20px';
elemento.style.backgroundColor = 'blue';

// Preferir (classes CSS)
elemento.classList.add('destaque');
```

Define os estilos no CSS e apenas adiciona/remove classes em JavaScript. Isto mantém a separação de responsabilidades.

5 Procedimento

5.1 Tarefa 1: Preparar Ambiente

1. Crie uma pasta nova para este laboratório (por exemplo, lab05-lista-compras) e abra essa pasta no Visual Studio Code
2. Crie a seguinte estrutura de pastas e ficheiros:
 - lista-compras.html
 - styles/lista-compras.css
 - scripts/lista-compras.js
3. No ficheiro lista-compras.html, adicione a estrutura HTML básica:

```
<!DOCTYPE html>
<html lang="pt">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Lista de Compras</title>
  <link rel="stylesheet" href="styles/lista-compras.css">
  <script src="scripts/lista-compras.js" defer></script>
</head>
<body>
  <div class="container">
    <h1>Lista de Compras</h1>

    <div class="adicionar-item">
      <input type="text" id="input-item" placeholder="Adicionar item...">
      <button id="btn-adicionar">Adicionar</button>
    </div>

    <ul id="lista-itens">
      <!-- Itens serao adicionados aqui dinamicamente -->
    </ul>
```

```
<div class="info">
  <p>Total de itens: <span id="total-itens">0</span></p>
</div>
</body>
</html>
```

4. No ficheiro `styles/lista-compras.css`, adicione os estilos em 3 ações curtas:

Ação 1 - Base da página

```
body {
  font-family: Arial, sans-serif;
  background: #f4f4f4;
  padding: 20px;
}

.container {
  background: white;
  border: 1px solid #ddd;
  border-radius: 8px;
  padding: 20px;
  max-width: 500px;
  margin: 0 auto;
}

h1 {
  text-align: center;
  margin-bottom: 20px;
}
```

Ação 2 - Input e botões

```
.adicionar-item {
  display: flex;
  gap: 10px;
  margin-bottom: 20px;
}

#input-item {
  flex: 1;
  padding: 10px;
  border: 1px solid #ccc;
  border-radius: 4px;
}

button {
  padding: 10px 14px;
  background: #2b7a78;
  color: white;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}
```

```
button:hover {  
    background: #205e5d;  
}
```

Ação 3 - Lista e contador

```
#lista-itens {  
    list-style: none;  
    margin-bottom: 20px;  
}  
  
.item-lista {  
    display: flex;  
    justify-content: space-between;  
    align-items: center;  
    padding: 10px;  
    border-bottom: 1px solid #eee;  
}  
  
.item-nome {  
    flex: 1;  
}  
  
.btn-remover {  
    background: #c0392b;  
}  
  
.btn-remover:hover {  
    background: #992d22;  
}  
  
.info {  
    text-align: center;  
    margin-top: 15px;  
}  
  
#total-itens {  
    font-weight: bold;  
}
```

5. No ficheiro `scripts/lista-compras.js`, adicione o código JavaScript

6. Adicione um `console.log` para testar:

```
console.log("Lista de Compras carregada!");
```

7. Abra `lista-compras.html` no browser e verifique que a página carrega corretamente

5.2 Tarefa 2: Lista de Compras - Versão Simples

Vamos implementar a funcionalidade básica: adicionar e remover itens.

5.2.1 Passo 1: Estrutura de Dados

No ficheiro `lista-compras.js`, crie um array para guardar os itens:

```
// Array para guardar os itens da lista
let itens = [];

// Funcao para adicionar item
function adicionarItem(nome) {
    // Adicionar ao array
    itens.push(nome);

    console.log("Item adicionado:", nome);
    console.log("Lista completa:", itens);
}

// Testar (temporario)
adicionarItem("Leite");
adicionarItem("Pao");
```

Recarregue a página e verifique a Console. Deve ver os itens a serem adicionados ao array.

5.2.2 Passo 2: Renderizar Lista no DOM (sem remover ainda)

Agora vamos criar uma função que desenha a lista no HTML:

```
// Funcao para renderizar lista no DOM
function renderizarLista() {
    // Selecionar elemento ul
    let listaElemento = document.getElementById('lista-itens');

    // Limpar lista
    listaElemento.innerHTML = '';

    // Percorrer array de itens (for tradicional)
    for (let indice = 0; indice < itens.length; indice++) {
        let item = itens[indice];

        // Criar elemento li
        let li = document.createElement('li');
        li.classList.add('item-lista');

        // Criar span para o nome
        let span = document.createElement('span');
        span.classList.add('item-nome');
        span.textContent = item;

        // Neste passo, mostramos apenas o nome
        li.appendChild(span);

        // Adicionar li a lista
        listaElemento.appendChild(li);
    }

    // Atualizar contador
    atualizarTotal();
}
```

```
// Funcao para atualizar total de itens
function atualizarTotal() {
    let totalElemento = document.getElementById('total-itens');
    totalElemento.textContent = itens.length;
}

// Modificar funcao adicionarItem para renderizar
function adicionarItem(nome) {
    itens.push(nome);
    renderizarLista(); // Renderizar apos adicionar
}
```

Recarregue a página. Deve ver os itens aparecerem na lista e o contador deve atualizar.

5.2.3 Passo 3: Adicionar funcionalidade de remover

Agora vamos acrescentar o botão Remove sem reescrever a função toda. Adicione os 3 trechos em separado, exatamente nos pontos indicados.

Trecho 1 - criar e configurar o botão

Este trecho cria o botão, define o texto e associa o clique à função de remoção. **Local:** dentro do for de renderizarLista(), imediatamente depois de criar o span.

```
let btnRemover = document.createElement('button');
btnRemover.classList.add('btn-remover');
btnRemover.textContent = 'Remover';
btnRemover.onclick = function() {
    removerItem(indice);
};
```

Trecho 2 - adicionar o botão ao item da lista

Este trecho coloca o botão dentro do li, para aparecer ao lado do nome. **Local:** ainda dentro do for, antes da linha listaElemento.appendChild(li).

```
li.appendChild(btnRemover);
```

Trecho 3 - criar a função de remoção

Este trecho remove o item pelo índice e volta a renderizar a lista para atualizar o ecrã. **Local:** fora da função renderizarLista(), ao mesmo nível das outras funções (ex.: adicionarItem, atualizarTotal).

```
function removerItem(indice) {
    itens.splice(indice, 1);
    renderizarLista();
}
```

Teste novamente: agora cada linha deve ter o botão "Remover" funcional.

5.2.4 Passo 4: Conectar ao Input

Agora vamos conectar o input e botão:

```
// Remover testes anteriores (adicionarItem("Leite"), etc.)

// Selecionar elementos do formulario
let inputItem = document.getElementById('input-item');
let btnAdicionar = document.getElementById('btn-adicionar');
```

```
// Event listener para botao adicionar
btnAdicionar.addEventListener('click', function() {
    let nome = inputItem.value.trim();

    // Validar input
    if (nome === '') {
        alert('Por favor, insira um nome para o item!');
        return;
    }

    // Adicionar item
    adicionarItem(nome);

    // Limpar input
    inputItem.value = '';

    // Focar input novamente
    inputItem.focus();
});

// Event listener para tecla Enter no input
inputItem.addEventListener('keypress', function(evento) {
    if (evento.key === 'Enter') {
        btnAdicionar.click(); // Simular clique no botao
    }
});
```

5.2.5 Passo 5: Testar

1. Recarregue a página
2. Digite um nome no input e clique "Adicionar"
3. Adicione vários itens
4. Experimente remover itens
5. Verifique que o total atualiza corretamente
6. Experimente pressionar Enter em vez de clicar no botão

5.3 Tarefa 3: Funcionalidades Extra

Vamos adicionar mais funcionalidades úteis tirando partido da manipulação do DOM.

5.3.1 Passo 1: Botão Limpar Tudo

Adicionar botão no HTML em `lista-compras.html`:

```
<div class="info">
  <p>Total de itens: <span id="total-itens">0</span></p>
  <button id="btn-limpar">Limpar Tudo</button>
</div>
```

Adicionar funcionalidade JavaScript:

```
// Event listener para limpar tudo
let btnLimpar = document.getElementById('btn-limpar');
btnLimpar.addEventListener('click', function() {
  if (itens.length === 0) {
    alert('A lista ja esta vazia!');
    return;
  }

  if (confirm('Tem a certeza que quer limpar toda a lista?')) {
    itens = [];
    renderizarLista();
  }
});
```

Adicionar estilo CSS:

```
#btn-limpar {
  margin-top: 15px;
  width: 100%;
  background: #e74c3c;
}

#btn-limpar:hover {
  background: #c0392b;
}
```

5.3.2 Passo 2: Ordenar Lista

Adicionar botão de ordenação no HTML (com uma nova zona de controlos em do anterior botão único de limpar):

```
<div class="controlos">
  <button id="btn-ordenar">Ordenar A-Z</button>
  <button id="btn-limpar">Limpar Tudo</button>
</div>
```

Adicionar CSS para o layout:

```
.controlos {
  display: flex;
  gap: 10px;
  margin-top: 15px;
}

.controlos button {
  flex: 1;
}

#btn-ordenar {
  background: #27ae60;
}

#btn-ordenar:hover {
  background: #229954;
}
```

Implementar a funcionalidade de ordenação:

```
// Event listener para ordenar
let btnOrdenar = document.getElementById('btn-ordenar');
btnOrdenar.addEventListener('click', function() {
  // Ordenar array alfabeticamente
  itens.sort(function(a, b) {
    return a.localeCompare(b);
  });

  renderizarLista();

  console.log("Lista ordenada!");
});
```

5.3.3 Passo 3: Desafio - Tema Claro/Escuro

Adicionar checkbox no HTML:

```
<div class="tema-toggle">
  <label for="toggle-escuro">Modo Escuro:</label>
  <input type="checkbox" id="toggle-escuro">
</div>
```

Adicionar CSS (pode colocar num ficheiro à parte ou no fim de lista-compras.css):

```
.tema-toggle {
  margin-bottom: 20px;
  text-align: center;
}

body.escuro {
  background: linear-gradient(135deg, #2c3e50 0%, #34495e 100%);
}

body.escuro .container {
  background: #34495e;
  color: #ecf0f1;
}

body.escuro h1 {
  color: #ecf0f1;
}

body.escuro .item-lista {
  background: #2c3e50;
}

body.escuro .item-nome {
  color: #ecf0f1;
}
```

Implementar a funcionalidade:

```
// Toggle tema
let toggleEscuro = document.getElementById('toggle-escuro');
toggleEscuro.addEventListener('change', function() {
```

```
document.body.classList.toggle('escuro');  
});
```

Depois de implementar o `toggle` do tema e os estilos `body.escuro`, o resultado esperado deve ser semelhante ao da Figura 2.



Figura 2: Resultado esperado após ativar o modo escuro

Seletores CSS: espaço vs sem espaço

Tenha particular atenção aos espaços nos seletores CSS. Quando escreve `body.escuro`, está a referir-se ao elemento `<body>` que tem a classe `escuro`. Se escrever `body .escuro`, com espaço, está a referir-se a qualquer elemento com a classe `escuro` que esteja *dentro* do `body`. O mesmo acontece, por exemplo, com `.item-lista.ativo` (o mesmo elemento tem as duas classes) e `.item-lista .ativo` (um elemento com classe `ativo` dentro de outro com classe `item-lista`).

6 Conceitos Importantes Revisitados

6.1 Event Delegation

Para elementos criados dinamicamente, existem duas formas comuns de tratar o clique no botão `Remover`.

1. Um listener em cada botão

Foi esta a abordagem usada na implementação principal do laboratório. Sempre que a lista é renderizada, cada botão recebe o seu próprio onclick.

```
function renderizarLista() {
  let listaElemento = document.getElementById('lista-itens');
  listaElemento.innerHTML = '';

  for (let indice = 0; indice < itens.length; indice++) {
    let item = itens[indice];

    let li = document.createElement('li');
    li.classList.add('item-lista');

    let span = document.createElement('span');
    span.classList.add('item-nome');
    span.textContent = item;

    let btnRemover = document.createElement('button');
    btnRemover.classList.add('btn-remover');
    btnRemover.textContent = 'Remover';
    btnRemover.onclick = function() {
      removerItem(indice);
    };

    li.appendChild(span);
    li.appendChild(btnRemover);
    listaElemento.appendChild(li);
  }

  atualizarTotal();
}
```

Como funciona: cada botão fica logo associado à sua ação. É a solução mais simples de compreender e, por isso, adequada nesta fase.

2. Event delegation

Nesta abordagem, os botões continuam a ser criados dinamicamente, mas o listener fica no elemento pai ul. O pai recebe o clique e verifica em que elemento ele aconteceu.

```
function renderizarLista() {
  let listaElemento = document.getElementById('lista-itens');
  listaElemento.innerHTML = '';

  for (let indice = 0; indice < itens.length; indice++) {
    let item = itens[indice];

    let li = document.createElement('li');
    li.classList.add('item-lista');

    let span = document.createElement('span');
    span.classList.add('item-nome');
    span.textContent = item;

    let btnRemover = document.createElement('button');
    btnRemover.classList.add('btn-remover');
    btnRemover.textContent = 'Remover';
    btnRemover.dataset.indice = indice;
```

```
        li.appendChild(span);
        li.appendChild(btnRemover);
        listaElemento.appendChild(li);
    }

    atualizarTotal();
}

document.getElementById('lista-itens').addEventListener('click',
function(evento) {
    if (evento.target.classList.contains('btn-remover')) {
        let indice = Number(evento.target.dataset.indice);
        removerItem(indice);
    }
});
```

Como funciona neste caso:

- O utilizador clica num botão Remover
- O evento sobe do botão para o elemento pai ul
- O listener do ul verifica se o elemento clicado tem a classe btn-remover
- Se tiver, lê o valor de data-indice e chama removerItem(indice)

Quando usar event delegation

Para este laboratório, a primeira abordagem é melhor porque é mais direta e mais fácil de seguir. A **event delegation** torna-se especialmente útil quando existem muitos elementos dinâmicos ou quando queremos centralizar o tratamento de eventos num único ponto.

6.2 Template Literals vs createElement

Há duas formas de criar HTML dinamicamente:

1. createElement (o que usámos):

```
let li = document.createElement('li');
li.textContent = 'Item';
lista.appendChild(li);
```

2. innerHTML com template literals:

```
lista.innerHTML += `<li>Item</li>`;
```

createElement é mais seguro e performante. **innerHTML** é mais simples mas pode ter problemas de segurança (XSS) se inserir dados não validados.

7 Resultado Esperado

No final deste laboratório, deve ter:

- ☐ Lista de Compras funcional com interface bonita
- ☐ Capacidade de adicionar e remover itens

- ☐ Funcionalidades de ordenação e limpeza
- ☐ Tema claro/escuro funcionando
- ☐ Código JavaScript bem organizado e comentado
- ☐ Compreensão profunda de manipulação do DOM
- ☐ Event listeners configurados adequadamente
- ☐ Capacidade de criar elementos dinamicamente

8 Questões para Reflexão

1. Qual a diferença entre `querySelector` e `querySelectorAll`?
2. Porque usar `classList` em vez de manipular `style` diretamente?
3. Qual a vantagem de remover e recriar elementos vs simplesmente escondê-los com CSS?
4. Porque limpar o `innerHTML` antes de renderizar a lista?
5. Como funcionam os event listeners em elementos criados dinamicamente?

9 Próximos Passos

Para evoluir este exercício de forma autónoma, pode aprofundar JavaScript focando em:

- Estruturas de dados: arrays e objetos
- Arrays de objetos e manipulação complexa
- JSON (JavaScript Object Notation)
- Persistência de dados
- Importação e exportação de dados
- Aplicar estruturas ao projeto da Lista de Compras

Recursos recomendados para estudo:

- MDN Web Docs - DOM: https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model
- JavaScript.info - DOM: <https://javascript.info/document>
- MDN - Array Methods: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array

10 Referências

- **MDN Web Docs - DOM:** https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model
- **JavaScript.info - DOM:** <https://javascript.info/document>
- **MDN - Arrays:** https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array
- **MDN - createElement:** <https://developer.mozilla.org/en-US/docs/Web/API/Document/createElement>
- **MDN - classList:** <https://developer.mozilla.org/en-US/docs/Web/API/Element/classList>
- **Eloquent JavaScript - DOM:** https://eloquentjavascript.net/14_dom.html